StatARPLS: An R Package for Baseline Correction of Spectra with Automatic Smoothing and Uncertainty

Taruna Dagur
School of Engineering and Computer Science, Victoria University of Wellington
tardagur@gmail.com

***Abstract*—Spectra from techniques such as infrared and Raman spectroscopy contain an unwanted, slowly changing background called the baseline, which has to be removed before the peaks can be analysed. This report describes how StatARPLS, an R package that implements asymmetrically reweighted penalized least squares from first principles. The package adds two things, automatic choice of the smoothing parameter by generalised cross-validation, and a bootstrap stability band that shows how much the fitted baseline moves with the noise. The S3 object containing print, summary, plot, predict, and compare methods for the fitting engine is described in detail. This includes an overview of the design decisions and the testing/peer review process, the problems encountered, and results on simulated and real spectra data. The report is completed by a critique of the work and the points that would have to be changed if the work was done all over again from the start.**

Keywords*—*baseline correction, spectroscopy, R package, cross-validation, bootstrap, reproducibility

I. Rationale

The features of a measured spectrum are a set of peaks carrying the chemical information and a smooth background, called the baseline. The true baseline is not determined and has to be estimated and subtracted from the data. Asymmetrically reweighted penalised least squares (ARPLS) [1] is a method for smooth baseline estimation. The peaks are forced not to exceed the estimated background by assigning low weights to such data points above the estimated baseline. The main parameter of ARPLS is the smoothing parameter lambda. The flexibility of the fit is determined by this parameter. If lambda is chosen too small the baseline follows the tops of the peaks, and if it is chosen too large, parts of the background are missed. The best value also changes from one data set to another [2].

Existing R tools leave users to guess lambda and return a single baseline without indicating how reliable it is. The aim of this project was therefore to build a tested and documented R package, called StatARPLS, that implements ARPLS from first principles, selects lambda automatically, and reports an uncertainty band around the baseline. This report outlines the development of the package in detail and recaps some of the key lessons learned along the way. As the vignette outlines how to use the package, the rest of this report is organised with the rationale for the package, its requirements, implementation, tools, results, challenges and conclusions from the project. An image of the project mind map is provided in Fig. 1.

The method the project is based on, ARPLS, is a good basis for several reasons. ARPLS is a statistical method that creates a smooth curve through the data points that lie on the baseline. No manual drawing of lines under the spectrum has to be done. The weights of ARPLS are calculated during the iterations of the algorithm using the mean and the standard deviation of the data points below the currently considered part of the baseline. Thus, ARPLS needs fewer parameters than methods of similar kind like asymmetric least squares that needs to specify an asymmetry. Each step of ARPLS is a simple linear solve. The major drawback of ARPLS is that lambda has to be chosen by hand, which is the problem that StatARPLS tries to solve.

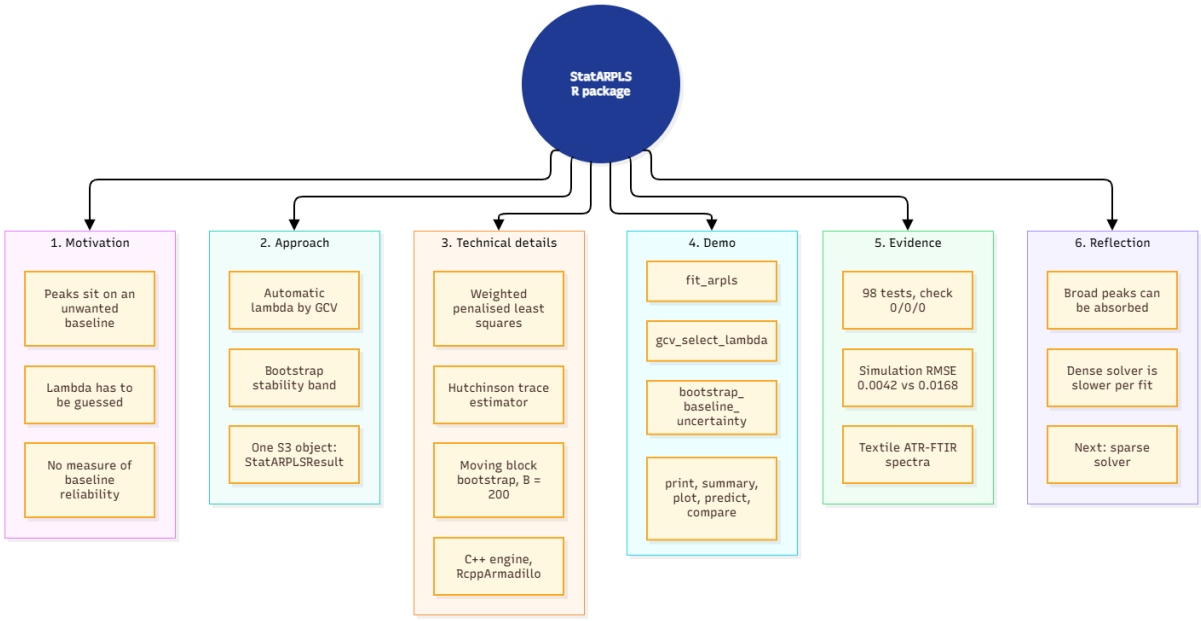


Fig. 1. Overview of the StatARPLS project, from the motivation to the reflection.

II. Requirements and Feedback

A. Feedback That Changed the Design

Feedback on my design plan changed two things. First, lambda had to be chosen with a generalised cross-validation (GCV) score [3], using Hutchinson's stochastic trace estimator [4] for the effective degrees of freedom, and not with a loop that scores each lambda using the ARPLS objective itself. Second, the uncertainty had to come from a moving-block bootstrap [5] and not from resampling residuals one at a time. The residuals of a spectrum are strongly correlated, so independent resampling would give a band that is too narrow. The lecturers also asked me to test on two kinds of real data, and to compare only with statistical methods from the existing baseline package, such as asymmetric least squares (AsLS) [6], IRLS and polynomial fitting.

III. Implementation

A. Structure of the Package

I developed the package with devtools and usethis [7], using a repeating cycle of load, document, test and check, with the code kept on GitHub. The code is split into four R files and one C++ file (Fig. 2). The file constructors.R holds the result class, fit-arpls.R holds the ARPLS fit, parameter-selection. R holds GCV and the bootstrap, methods.R holds the S3 methods, and arpls_solver.cpp holds the compiled solver. In Fig. 2, an arrow means "uses", and all arrows point one way, so there are no circular dependencies. The S3 methods only depend on the shape of the result object and never call the fitting code.

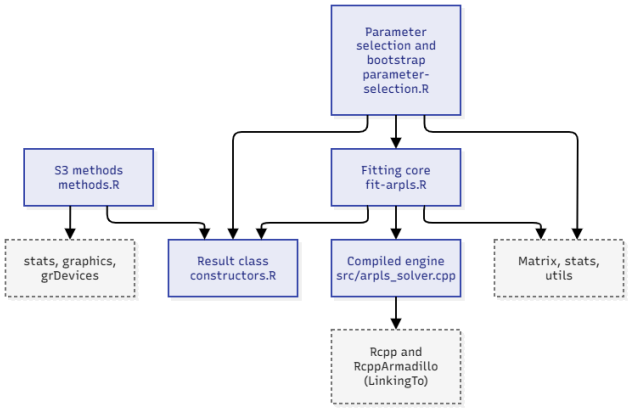


Fig. 2. Module view of the package. Boxes are files and an arrow means "uses". Dashed boxes are outside packages.

Fig. 3. Every function returns one object of class StatARPLSResult with ten fields: the signal, the wavenumber axis, the baseline, the corrected signal, the residual variance, the stability bounds, lambda, the number of iterations, a convergence flag and the call. Extra information, such as the GCV table and the bootstrap details, is stored as attributes, so the class stays the same whichever function made it. Fig. 3 shows what happens when the package runs. In part A, the axis is validated, every candidate lambda is fitted and scored, and the result is built. In part B, the noise residuals are resampled, the baseline is refitted for each resample, and the band is summarised.

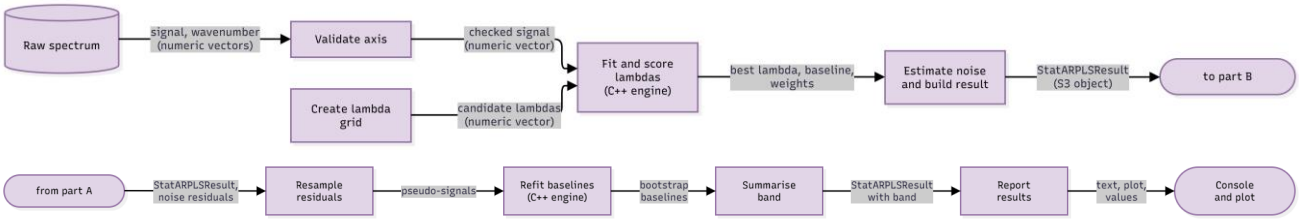


Fig. 4. Run-time data flow. Part A (top) selects lambda and fits the baseline. Part B (bottom) builds the stability band.

B. Design Decisions

Several smaller decisions shaped how the package behaves.

- S3 classes was chosen because it is simple, commonly used in R, and its easy for others to extend afterwards.
- During the bootstrapping the lambda was kept fixed for every resample, so the uncertainty band describes how much the baseline varies for the chosen smoothing, not how unsure we are about lambda itself.
- Failed resamples are counted. If a refit does not converge it is dropped and counted, and the number is stored with the other bootstrap details, so nothing disappears silently.
- compare() has its own generic. It was defined with UseMethod() so that it does not hide an existing base R function.
- The plotting method uses base graphics, avoiding a dependency on ggplot2 and keeping package dependencies to a minimum..
- The functions check that the values are finite and that the wavenumber axis is strictly increasing, with a clear message that tells the user to sort it. They warn when the spacing is uneven.
- The package warns when GCV selects the smallest lambda in the search grid, because a very small lambda can be problematic because it allows the baseline to bend too much and become overly flexible.

C. The Methods in Brief

For a fixed lambda, ARPLS looks for a baseline z that stays close to the signal y but is smooth, as in Eq. (1). Here w is a weight for each point and Δ^2 is the second difference, which measures how much the curve bends.

$$\min_z \sum_i w_i (y_i - z_i)^2 + \lambda \sum_i (\Delta^2 z_i)^2 \quad (1)$$

Setting the derivative to zero gives the linear system in Eq. (2), which is the step I solve in C++. W is the diagonal matrix of weights and D is the second-difference matrix.

$$(W + \lambda D^\top D)\, z = W y \quad (2)$$

After each solve the weights are updated from the residuals $r = y - z$, as in Eq. (3). Here m and s are the mean and standard deviation of the negative residuals. A point far above the baseline gets a weight close to zero, so peaks stop pulling the baseline upwards. The two steps repeat until the weights stop changing.

$$w_i = \frac{1}{1 + \exp\left(\dfrac{2\,(r_i - (2s - m))}{s}\right)} \tag{3}$$

To choose lambda, GCV scores every value in a grid with Eq. (4). The top measures how far the fit is from the data and the bottom charges for flexibility, where $H = (W + \lambda D^{\top} D)^{-1} W$ and tr H is the effective degrees of freedom. This trace is too slow to compute exactly for every lambda, so I estimate it with m = 30 random vectors u of plus and minus one, using Hutchinson's estimator [4] in Eq. (5).

$$\mathrm{GCV}(\lambda) = \frac{n \sum_i w_i\, r_i^2}{(n - \operatorname{tr} H_\lambda)^2} \tag{4}$$

$$\operatorname{tr} H_\lambda \approx \frac{1}{m} \sum_{k=1}^{m} u_k^{\top} H_\lambda u_k \tag{5}$$

For the bootstrap, I take blocks of residuals from the points classified as baseline, add them to the fitted baseline, refit with the same lambda, and repeat 200 times by default. The band is centred on the fitted baseline and uses the 2.5% and 97.5% quantiles of the refits.

D. The Compiled Engine

The linear solve is the slowest step, so it was written in C++ with RcppArmadillo. A test checks that the two agree to within 0.00001, and on a 500-point spectrum the compiled engine is about 20 times faster (between 14 and 22 times in the runs).

E. Testing and Checking

It was worked in nine gated steps. It was not started until the next step devtools::check() gave 0 errors, 0 warnings and 0 notes. The tests use testthat, and there are 98 of them, all passing. They cover input checks, GCV, the block structure of the bootstrap, whether the seeded functions leave the user's random number state alone, the S3 methods, and odd inputs such as missing values and flat spectra. Table I shows how each part of the package was checked.

TABLE I. PARTS OF THE PACKAGE AND HOW EACH WAS CHECKED

Part	File	How it was checked
Result class	constructors.R	Tests of valid and invalid input
ARPLS fit	fit-arpls.R	Tests for missing values, flat spectra and the axis checks
Compiled engine	arpls_solver.cpp	Parity test against the pure R engine
GCV and bootstrap	parameter-selection.R	Tests for the grid, the block structure and the seed state
S3 methods	methods.R	Tests for each method, plus the peer review fixes

F. Peer Review

My first test plan was peer reviewed, and the review gave 17 findings. I fixed 13 of them, clarified 3 in the plan, and deferred 1 (the fixed default values in fit_arpls(), which are documented and can be changed). Examples of fixes are: the plot legend still showed the band when it was switched off; predict() with the spline method ignored the rule argument; compare() silently dropped a bad argument; fit_arpls() crashed on missing values and on a flat spectrum; and R CMD check complained about a badly named file and undocumented arguments. The updated test plan lists each finding and what I did about it. Some findings were about my instructions and not my code. The plan did not say that the remotes package is needed, did not say whether each command goes in the R console or in a terminal, and gave a test count that did not match the reviewer's. This taught me that a plan has to be tried on another computer before it can be trusted.

IV. TOOLS AND REPRODUCIBILITY

The package development took place on Windows with R 4.5.2, using devtools version 2.5.2, remotes version 2.5.0 and Rtools (GCC 14.3.0). A Windows binary for version 0.1.0 was also built with devtools::build(binary = TRUE) so that users without Rtools can install the package without having to compile the C++ functions of the package. In every example and script the random number generation was set to a fixed seed value: 501 for the simulated spectrum and 1 and 2 for GCV and the bootstrap, respectively. The functions of the package also restore the random number generation to the user's previous state after execution of the function so that the execution of a function does not affect any subsequent random number generation.

All exported functions are documented with roxygen comments. Every argument has a parameter description and the return value is described as a list of its parts. The examples are short and run in a few seconds. Files that are only for development, such as the data-raw scripts, the presentation and the test plan, are left out of the built package with anchored entries in . Rbuildignore, for example ^data-raw$. Besides the standard base packages, the package only imports Rcpp, RcppArmadillo and Matrix, and every declared import is used, so R CMD check gives no note about unused imports. The README gives the install steps, a short example and the known limits.

Three kinds of data were used (Table II). The simulated spectra are generated by code. The ChemoSpec data set SrE.IR [8] has 16 infrared spectra of saw palmetto extracts and reference oils, with 1,868 points each. The third is a public data set of 160 infrared spectra of textile fibres from Mendeley Data [9], licensed CC BY 4.0. It uses its raw feature matrix and two changes were made: transmission was converted to absorbance and every second point was kept, which gives 1,726 points between 4000 and 550 cm^{-1}. The textile data are not stored in the repository. The script data-raw/make-textile-figures.R rebuilds the figures from the downloaded file and prints which spectra were used (sample IDs MTA_ALP_073, MTA_COT_062, MTF_PES_141 and MTF_ACR_005). The repository also holds the README, the vignette, the updated test plan, the presentation and this report. Negative results are reported in Sections V and VI, not hidden.

TABLE II. DATA SETS USED IN THIS PROJECT

Data set	Spectra	Points	Use
Simulated	20	1,868	Error against a known baseline
SrE.IR (ChemoSpec) [8]	16	1,868	Real infrared data, sensitivity to lambda
Textile fibres [9]	160 (4 shown)	1,726	Failure case and the warning

V. Results

First it was, checked with the package on a simulation where the true baseline is known: 20 spectra with 1,868 points, a smooth curved baseline, 8 narrow peaks and noise with standard deviation 0.01. Table III gives the median error (RMSE) against the true baseline. StatARPLS with automatic lambda had the lowest error on all 20 spectra, about four times lower than the best of three fixed AsLS settings. AsLS was not tuned: Three fixed values of lambda were used, so a tuned AsLS would probably be closer. Fig. 4 shows one simulated spectrum with its fitted baseline. The baseline passes under both peaks and follows the slow curve.

The summary() method prints the main results of the fit, as in the example below for this spectrum.

```
Lambda:          316228
Converged:       TRUE in 4 iterations
Residual variance:   1.969
Mean stability-band width: 0.7544
--- GCV Parameter Selection ---
Candidates evaluated: 13
Effective df:    7.369
--- Residual Bootstrap ---
Replicates: 100 (0 failed)
Mean pointwise SE: 0.2021
```

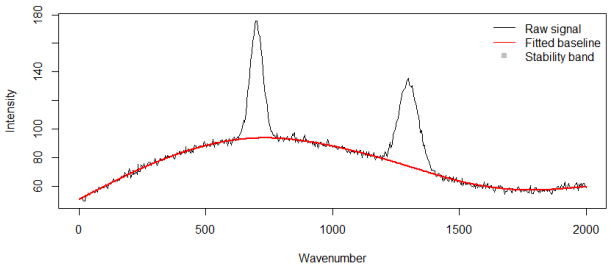


Fig. 5. One simulated spectrum (raw signal in black) with the baseline fitted by StatARPLS (red). Lambda was chosen automatically by GCV (316,228) and the stability band was too thin to see.

TABLE III. Median baseline error in the simulation

Method	Median RMSE
StatARPLS (automatic lambda)	0.0042
AsLS, lambda = 10^7	0.0168
AsLS, lambda = 10^5	0.0288
Polynomial fit (degree 4)	0.0625
IRLS (default)	0.0744
AsLS, lambda = 10^3	0.1065

On the 16 real infrared spectra, the AsLS baseline moved by a median of 30% of the signal range when lambda changed from 10^3 to 10^7, so the answer depends on the guess. StatARPLS chose different values of lambda for different spectra, from 3×10^5 to 10^8. Fig. 5 shows one of these spectra. The baseline stays under the sharp peaks near 1700 and 2900 cm^{-1} and follows the slow rise at low wavenumbers.

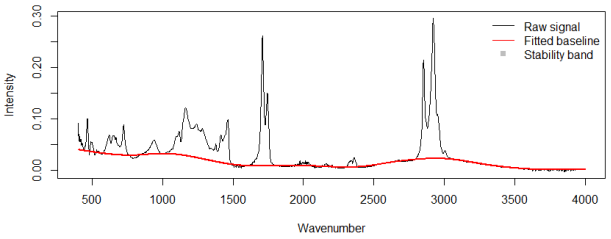


Fig. 6. One real infrared spectrum from the ChemoSpec data set SrE.IR with its fitted baseline (red). The stability band is too thin to see.

The textile spectra gave a negative result. With the default grid of lambda, GCV picked the smallest value (100) for the protein and polyester samples, and the baseline followed the peaks. The new warning fired on exactly those two. With a larger minimum lambda the baselines sat under the peaks (Fig. 6). The acrylic baseline still followed some broad bumps, and its band was very thin (0.0004), so a thin band does not mean a correct baseline.

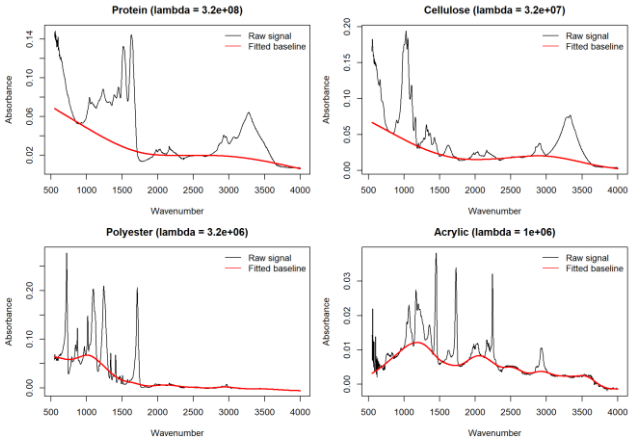


Fig. 7. Baselines for four textile fibres (protein, cellulose, polyester and acrylic) using a larger minimum lambda.

Fig. 7 shows the stability bands for the same four fits. The bands are only 0.0004 to 0.004 wide, so they are almost impossible to see. The acrylic band is the thinnest, although its baseline is not the best, so a thin band does not mean that the baseline is correct.

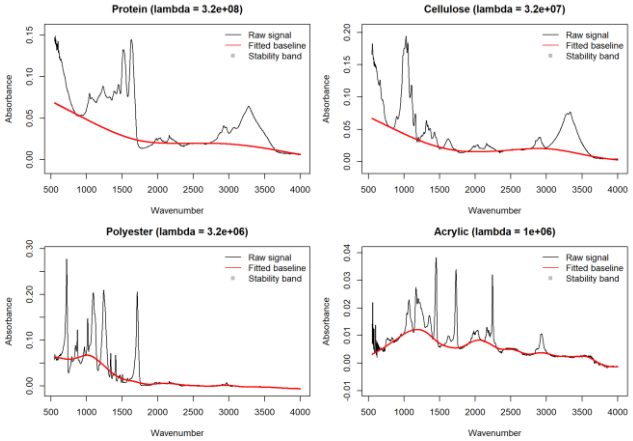


Fig. 8. Bootstrap stability bands for the four textile fits. The bands are too thin to see at this scale.

Table IV shows the run times on one spectrum. The time grows quickly with the number of points because the solve is dense. The AsLS method in the baseline package is faster per fit because it uses a sparse solver, so it does not claim that StatARPLS is faster than it. The compiled engine is faster than my own pure R version, but that is a comparison with my own code and not with other packages.

TABLE IV. RUN TIME FOR ONE SPECTRUM (DENSE SOLVER)

Points	Operation	Time
500	One fit, compiled engine	about 0.005 s
500	One fit, pure R engine	about 0.1 s
1,868	GCV over 13 values of lambda	about 1.7 s
1,868	Bootstrap, 100 resamples	about 16 s
3,092	GCV over 13 values of lambda	about 9.8 s

VI. CHALLENGES

These were the main problems I met, and what I did about them.

The compiled code was not being used. The file arpls_solver.cpp existed, but there was no RcppExports wrapper, so the package only ran the R loop. I regenerated the wrapper, fixed three defects in the C++ (error handling in the solve, a convergence rule that differed from the R version, and a missing margin for a near-zero scale), and added a test that the two engines agree.

The sparse solver did not build. The sparse solve with spsolve() needs SuperLU, which was not available in my Windows build. I went back to a dense solve and wrote the limit down as future work. This is why long spectra are slow.

Tests that could not fail. Two tests meant to force a singular system never did, because the system is always well behaved. They were replaced with tests that force a failure using mocked functions.

GCV can pick a lambda that is too small. On the textile spectra this made the baseline follow the peaks. The default grid was kept, added a warning when the smallest lambda is picked, and documented a larger grid as the workaround.

The first choice of data was not reachable. The plan was to use the RRUFF infrared library, but the site could not be reached, so it was switched to the Mendeley textile data set instead.

VII. SELF-REFLECTION AND CRITIQUE

TABLE V. REQUIREMENTS AND HOW WELL THEY WERE MET

Requirement	Status
One S3 class with five methods	Met
Compiled code with Rcpp	Met, but the solve is dense
Base graphics only	Met
Automatic lambda by GCV	Met, with a documented failure case
Moving-block bootstrap band	Met (200 resamples by default)
R CMD check without problems	Met (0 errors, 0 warnings, 0 notes)
Better performance or functionality	Partly: more functionality, not faster than baseline
Vignette, README and updated test plan	Met

Developing StatARPLS taught me how a statistical method becomes a real R package. The roxygen comments produce the help files and the NAMESPACE, the tests protect the behaviour, and R CMD check catches many problems that nobody sees when running code by hand. The main strength of the package is that it combines automatic choice of lambda, baseline correction and a stability band in one workflow. Checking after every step worked well, because devtools::check() had to be clean before I moved on, so problems were found while they were still small. The peer review also improved the package, because other people tried things I would not have tried. I also learned that a clean check does not prove that a package works: the missing compiled code and the tests that could not fail both passed the check. This taught me to keep what is delivered separate from what is only planned, and to test against a known truth, such as a simulation, as well as on real data where the truth is not known.

The package also has weaknesses. It does more than the tools I compared with, because it chooses lambda automatically and gives an uncertainty band, but it is not faster than the baseline package. The method does not work equally well on every spectrum: GCV can choose a lambda that is too small, as it did on the textile spectra, and broad peaks can be absorbed into the baseline. The stability band only shows how much the baseline moves with the noise at a fixed lambda. It does not include bias, so it can be thin when the baseline is wrong. My evaluation is also limited. The simulation has narrow peaks and a smooth baseline, where GCV works well, and it has no broad peaks. AsLS was not tuned, I only used two kinds of real infrared data, and the larger minimum lambda for the textile spectra was chosen after I saw the failure, so those fixed fits are not an unbiased test.

If I started again, I would change how I worked as well as the code. I would check the build tools on the first day and write a banded solver for the penalty matrix from the start, because that would make the cost grow in a straight line with the number of points. I would add a test on day one that shows the compiled code is really being called, and set up automatic checks .I would plan the evaluation earlier, by comparing with AsLS at its best lambda, including broad peaks in the simulation, and sorting out the real data . I would look for a selection rule that avoids a too-small lambda instead of using plain GCV with a warning.

The next steps are a banded or sparse solver, a comparison with AsLS at its best lambda, a selection rule that does not choose a too-small lambda. A larger real-data study would also help, in particular with broad peaks, where ARPLS can absorb a peak into the baseline.

VIII. CONCLUSION

StatARPLS is a tested and documented R package that implements ARPLS from first principles, chooses lambda by GCV, and reports a bootstrap stability band, in one S3 object. On simulated data it gave lower baseline error than untuned AsLS, and on real spectra it showed a failure case that it now warns about. The main limits are the dense solver and the narrow evaluation. The most useful lesson for me was to check the work at every step and to say clearly what is and is not delivered.

ACKNOWLEDGMENT

I thank the lecturers for their feedback on my design plan and the reviewer of my first test plan. I used the AI assistant Claude (Anthropic) during the project to help review and debug code, explain ideas, and draft documentation. I ran the code, checked the results and edited the text myself, and I am responsible for the content.

CODE AVAILABILITY

The source code of StatARPLS, the vignette, the tests, the presentation and this report are in the GitHub repository at https://github.com/tardagur/StatARPLS .

REFERENCES

[1] S.-J. Baek, A. Park, Y.-J. Ahn, and J. Choo, "Baseline correction using asymmetrically reweighted penalized least squares smoothing," *Analyst*, vol. 140, no. 1, pp. 250–257, 2015, doi: 10.1039/C4AN01061B.

[2] K. H. Liland, T. Almøy, and B.-H. Mevik, "Optimal choice of baseline correction for multivariate calibration of spectra," *Appl. Spectrosc.*, vol. 64, no. 9, pp. 1007–1016, 2010, doi: 10.1366/000370210792434350.

[3] P. Craven and G. Wahba, "Smoothing noisy data with spline functions: Estimating the correct degree of smoothing by the method of generalized cross-validation," *Numer. Math.*, vol. 31, no. 4, pp. 377–403, 1978, doi: 10.1007/BF01404567.

[4] M. F. Hutchinson, "A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines," *Commun. Statist. Simul. Comput.*, vol. 19, no. 2, pp. 433–450, 1990, doi: 10.1080/03610919008812866.

[5] H. R. Künsch, "The jackknife and the bootstrap for general stationary observations," *Ann. Statist.*, vol. 17, no. 3, pp. 1217–1241, 1989, doi: 10.1214/aos/1176347265.

[6] P. H. C. Eilers and H. F. M. Boelens, "Baseline correction with asymmetric least squares smoothing," Leiden Univ. Med. Centre, Leiden, The Netherlands, Tech. Rep., 2005.

[7] H. Wickham and J. Bryan, *R Packages*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2023.

[8] B. A. Hanson, "ChemoSpec: Exploratory chemometrics for spectroscopy," R package version 6.3.1, 2025. [Online]. Available: https://CRAN.R-project.org/package=ChemoSpec

[9] R. Parkar, A. Jain, M. Prendergast-Miller, T. Stanton, K. Sheridan, and M. Gallidabino, "A dataset of infrared (ATR-FTIR) spectra for textile fibres of natural and man-made origin," Mendeley Data, ver. 3, Feb. 2026, doi: 10.17632/rx3fjgz96x.3.